In this snippet, you've created an instance of Ruby's random number generator, and then replaced its rand method with one that returns a canned value. All its other methods will behave normally.

You can also use a partial double as a spy, using the expect(...).to have_received form you saw earlier:

```
>> allow(Dir).to receive(:mktmpdir).and_yield('/path/to/tmp')
=> #<RSpec::Mocks::MessageExpectation #<Dir (class)>.mktmpdir(any arguments)>
>> Dir.mktmpdir { |dir| puts "Dir is: #{dir}" }
Dir is: /path/to/tmp
=> nil
>> expect(Dir).to have_received(:mktmpdir)
=> nil
```

When you used a pure double as a spy, you had a choice of how to specify up front which messages the spy should allow. You could permit *any* message (using spy or as_null_object), or explicitly allow just the messages you want. With partial doubles, you can only do the latter. RSpec doesn't support the notion of a "partial spy," because it can't spy on all of a real object's methods in a performant way.

When you use partial doubles inside your specs, RSpec will revert all your changes at the end of each example. The Ruby object will go back to its original behavior. That way, you won't have to worry about the test double behavior leaking into other specs.

Since you are experimenting in stand-alone mode, you will need to call RSpec::Mocks.teardown explicitly to get this same cleanup to happen:

```
>> RSpec::Mocks.teardown
=> #<RSpec::Mocks::RootSpace:0x007ff2389bccb0>
>> random.rand
=> 0.9385928886462153
```

This call also exits from the stand-alone mode you've been experimenting in. If you want to keep exploring in the same IRB session, you'll need to call RSpec::Mocks.setup to go back into stand-alone mode.

Test Doubles Have Short Lifetimes



RSpec tears down all your test doubles at the end of each example. That means they won't play well with RSpec features that live outside the typical per-example scope, such as before(:context) hooks. You can work around some of these limitations with a method named with_temporary_scope.[1]

1. https://relishapp.com/rspec/rspec-mocks/v/3-6/docs/basics/scope